

INFORME DE ANÁLISIS ESTÁTICO DE CÓDIGO CON SONARQUBE



SANTIAGO ENRIQUE BASTIDAS ESTRADA

BIBIANA SOFIA CORTES MONTILLA

JOSE ALEJANDRO RODRIGUEZ DUEÑAS

GLENN ALEXANDER WARD ANTE

Laboratorio de Ingeniería de Software III

Profesor:

Daniel Eduardo Paz Perafan

Francisco Javier Obando Vidal

Juan Carlos Narvaez Narvaez

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

0. VIDEO

<https://youtu.be/iH8LQJqFy6Q>

1. DESPLIEGUE Y CONFIGURACIÓN DEL ENTORNO

Arquitectura de ejecución

El servidor de análisis estático se desplegó localmente mediante contenedorización utilizando Docker, ejecutando una instancia de SonarQube en el puerto 9000. El análisis del código se realizó desde un entorno Linux (WSL2 sobre Windows), lo cual permitió integrar correctamente las herramientas de compilación y análisis.

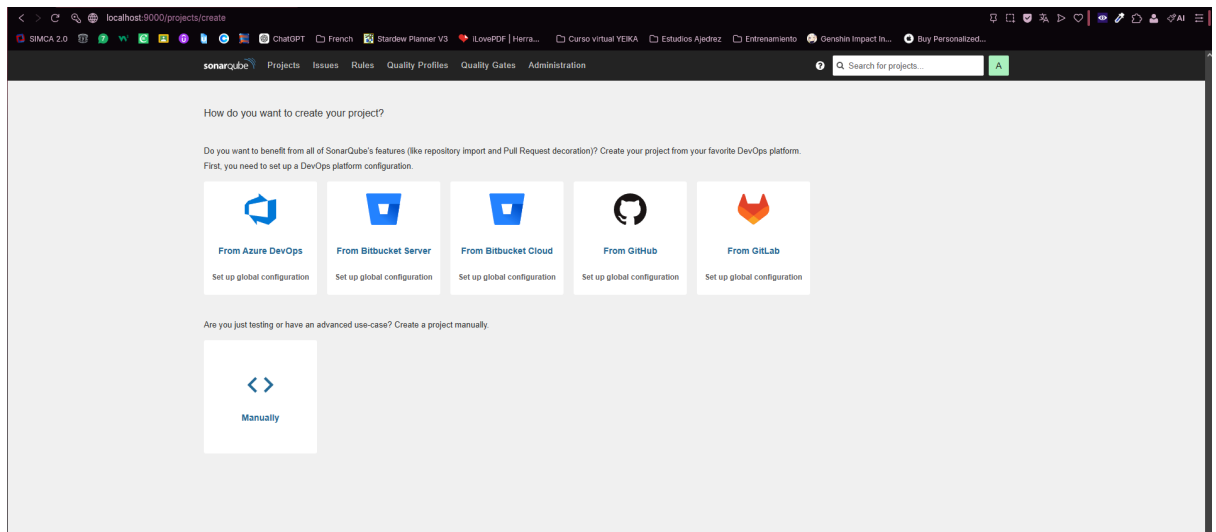
El flujo de ejecución fue el siguiente:

1. Levantamiento del servidor SonarQube mediante Docker.

Para iniciar el servidor de sonarqube, desde ubuntu y con ayuda de docker levantamos el siguiente comando:

```
sofia@popetaPC:~$ docker run -d --name sonarqube -p 9000:9000 sonarqube:lts
Unable to find image 'sonarqube:lts' locally
lts: Pulling from library/sonarqube
4f4fb700ef54: Pull complete
5d5a1fad7028: Pull complete
1df735f481ad: Pull complete
e24a8b9e652f: Pull complete
f3929ce9ef98: Pull complete
c7ad1fe61e07: Pull complete
eb27e3a98da1: Pull complete
60d98d907669: Pull complete
ce2bd68be4fc: Download complete
600f318704dc: Download complete
Digest: sha256:f709975ab31d2d08f5a3ae2dc73a31ee011afc8cf28845082c17c55d45df9df5
Status: Downloaded newer image for sonarqube:lts
5f2f6501c94d79138303da1cf943433d6146c224299be4c9a136f9819bd217dc
sofia@popetaPC:~$ |
```

Luego de esto ingresamos al servidor de sonarQube (<http://localhost:9000>) y creamos el proyecto en el que vamos a trabajar:



2. Compilación del backend en Java utilizando Maven: Dejamos todos los archivos que queremos que se escaneen listos, y para esto tenemos que compilar el backend.
3. Ejecución del escáner (sonar-scanner) desde la raíz del proyecto
Con el siguiente comando generamos el reporte:

C/C++

```
sonar-scanner -Dsonar.host.url=http://localhost:9000  
-Dsonar.login=squ_9345d743a272123277f174a06a674212147a2723A
```

```
INFO: Analysis report compressed in 100ms, zip size=239.5 kB  
INFO: Analysis report uploaded in 86ms  
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://localhost:9000/dashboard?id=Piedra-Azul  
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report  
INFO: More about the report processing at http://localhost:9000/api/ce/task?id=AZ35qbLRvxPTXkmubA4E  
INFO: Analysis total time: 10.830 s  
INFO: -----  
INFO: EXECUTION SUCCESS  
INFO: -----  
INFO: Total time: 11.706s  
INFO: Final Memory: 24M/108M  
INFO: -----  
sofia@popetaPC:~/PIEDRA-AZUL---SWIII-$ |
```

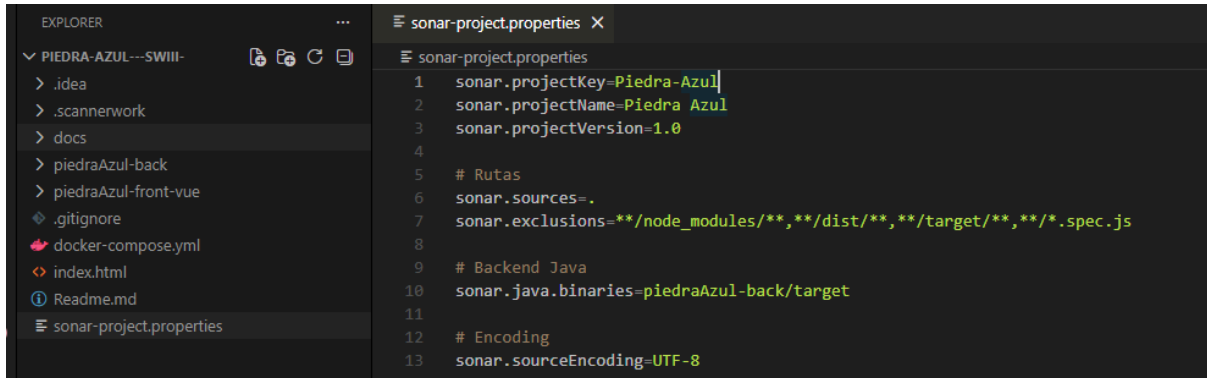
Ahora si vamos al link: <http://localhost:9000/dashboard?id=Piedra-Azul> podremos encontrar el reporte con toda la información del proyecto.

Configuración del escáner

El análisis se configuró mediante el archivo `sonar-project.properties` ubicado en la carpeta raíz del proyecto.

Con los siguientes parámetros principales:

- Definición del identificador del proyecto
- Inclusión de todo el código fuente
- Configuración de rutas de compilación para el backend Java
- Definición del encoding UTF-8



The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'PIEDRA-AZUL---SWIII-' with various folders and files. The code editor displays the 'sonar-project.properties' file with the following content:

```

1 sonar.projectKey=Piedra-Azul
2 sonar.projectName=Piedra Azul
3 sonar.projectVersion=1.0
4
5 # Rutas
6 sonar.sources=.
7 sonar.exclusions=**/node_modules/**,**/dist/**,**/target/**,**/*.spec.js
8
9 # Backend Java
10 sonar.java.binaries=piedraAzul-back/target
11
12 # Encoding
13 sonar.sourceEncoding=UTF-8

```

El análisis incluyó tanto el backend como el frontend. Sin embargo, el nivel de profundidad en el análisis del frontend es menor debido a limitaciones del soporte de SonarQube para archivos `.vue`, lo cual puede afectar la cantidad de hallazgos detectados en esta capa.

Exclusiones del análisis

Se definieron exclusiones específicas para evitar analizar código que no corresponde al desarrollo propio del equipo:

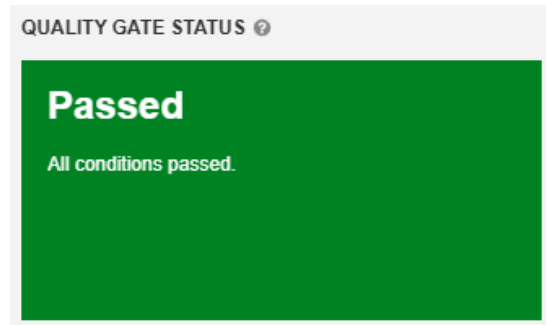
- `node_modules/` → dependencias externas del frontend
- `dist/` → archivos generados por el proceso de build
- `target/` → artefactos compilados del backend
- `*.spec.js` → archivos de pruebas

La inclusión de estos archivos generaría métricas incorrectas, ya que contienen código de terceros o código generado automáticamente, lo cual distorsiona la calidad real del software desarrollado, por eso mismo, no será tomado en cuenta para el análisis.

2. RESUMEN EJECUTIVO

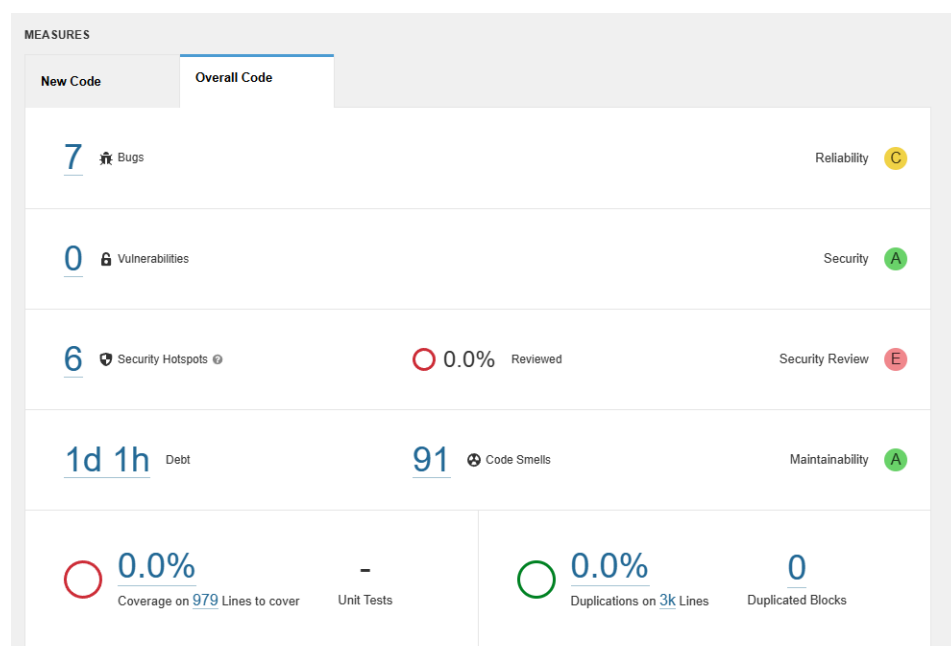
Estado final del proyecto

Resultado del Quality Gate: Según el reporte generado por sonarQube nuestro proyecto ha pasado.



Métricas globales

Metrica	Valor
Duplicación de código	0
Bugs	7
Vulnerabilidades	0
Code Smells	91



3. DESGLOSE DE LOS HALLAZGOS

Antes de entrar a los problemas específicos de front y back, voy a nombrar algunos problemas que se repiten bastante y que sería bueno que eventualmente los corregiremos:

- **Uso Inseguro de Opcionales en Reactividad:** Se detectaron múltiples instancias donde se accede al valor de un objeto (posiblemente un `ref` o un `computed` en Vue.js) sin verificar su existencia previa mediante `.isPresent()` o validaciones de nulidad.
- **Violación de Convenciones de Nomenclatura:** Diversos métodos no cumplen con el patrón de *camelCase* exigido por los estándares de desarrollo (ej. expresiones regulares como `^[a-z][a-zA-Z0-9]*$`).
- **Variables Locales Innecesarias:** Se identificaron variables como `admin` que son declaradas pero nunca utilizadas, lo que aumenta la carga cognitiva del código y ensucia el espacio de nombres local.

Capa Frontend (Vue / JavaScript)

En el sonarQube en el apartado de issues encontramos estos bogus relacionado a problemas en el front

The screenshot shows a list of three issues in SonarQube, all related to accessibility (WCAG 2-A) and missing table descriptions. Each issue is a 'Bug' of 'Minor' severity, 'Not assigned', and has a '5min effort' to resolve. The issues are located in the following files:

- `piedraAzul-front-vue/src/Views/Configuracion.vue` (L43)
- `piedraAzul-front-vue/src/Views/ListarCitas.vue` (L72)
- `piedraAzul-front-vue/src/Views/PortalPaciente.vue` (L118)

Each issue entry includes a checkbox to toggle details, a star icon, the severity level, a dropdown menu, the effort estimate, a comment link, and a link to the project's accessibility settings.

Problema 1: Ausencia de descripciones en tablas (Accessibility - WCAG 2-A)

The screenshot shows a detailed view of an accessibility issue in SonarQube. The issue is titled 'Add a description to this table.' and is located in the file `piedraAzul-front-vue/src/Views/ListarCitas.vue` at line 72. The issue is a 'Bug' of 'Minor' severity, 'Not assigned', and has a '5min effort' to resolve. The issue is related to the 'Web.TableWithoutCaptionCheck' rule. The code snippet shows a table element with the following structure:

```
<table>
  <thead>
    <tr>
      <th>#</th><th>Horas</th><th>Paciente</th><th>Documento</th>
      <th>Celular</th><th>Estado</th><th>Registrado por</th><th>Observación</th><th>Acción</th>
    </tr>
  </thead>
  <tbody>
    <tr v-for="(c, i) in citas" :key="c.id">
      <td class="num">{{ i + 1 }}</td>
    </tr>
  </tbody>
</table>
```

The issue is highlighted in a red box, and the text 'Add a description to this table.' is displayed above the table. The left sidebar shows a list of other issues, including several 'Code Smell' issues related to regular expressions and method names.

- **Descripción:** Se detectaron múltiples instancias en las vistas `Configuracion.vue` (L43), `ListarCitas.vue` (L72) y `PortalPaciente.vue` (L118) donde las tablas HTML carecen de un elemento descriptivo (`<caption>` o atributos `aria-describedby`).
- **Impacto:** Según lo que investigue esto significa un fallo de accesibilidad según las pautas WCAG 2.1, lo que implica que se impide que los usuarios que dependen de lectores de pantalla comprendan el contexto o propósito de la información tabular presentada.

Problema 2: Complejidad estructural en vistas principales

- **Descripción:** La distribución de los hallazgos en líneas avanzadas (como la L118 en `PortalPaciente.vue`) sugiere componentes con plantillas extensas.
- **Impacto:** El mantenimiento de componentes con cientos de líneas de código HTML aumenta el riesgo de errores de renderizado y dificulta la implementación de cambios rápidos en la interfaz de usuario.

Capa Backend (Java / Spring Boot)

A diferencia de la parte del front, en el sonarQube en el apartado de issues encontramos estos una gran cantidad de bugs y code smells relacionados a problemas en el back

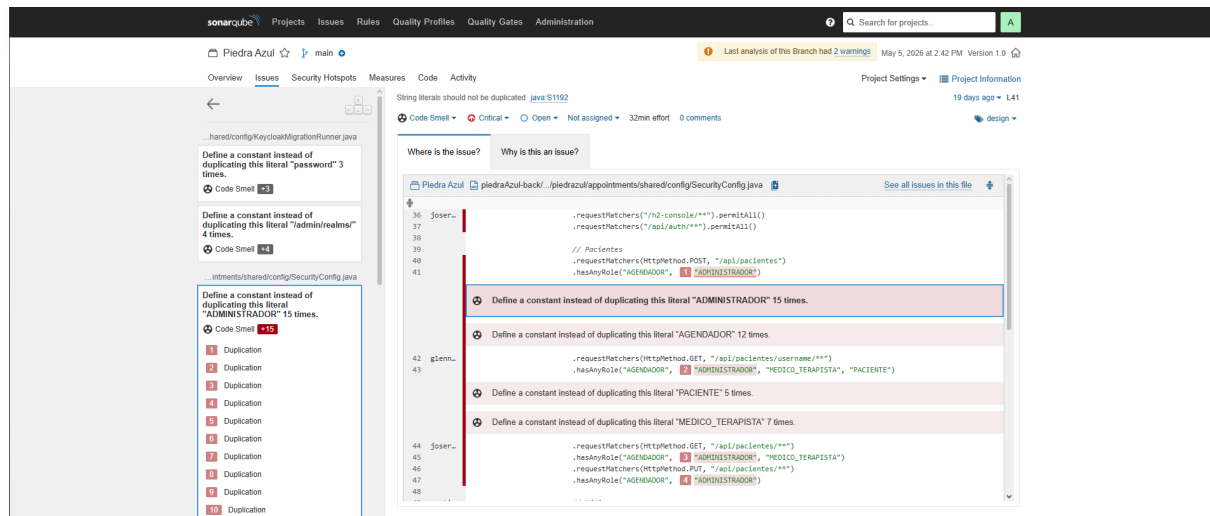
pedraAzul-back/.../appointments/services/MedicoTerapistaServiceTest.java	
☐ Call "resultado.isPresent()" before accessing the value. Bug Major Open Not assigned 10min effort Comment 1 month ago L88 cwe	
pedraAzul-back/.../appointments/services/PacienteServiceTest.java	
☐ Call "resultado.isPresent()" before accessing the value. Bug Major Open Not assigned 10min effort Comment 1 month ago L72 cwe	
☐ Call "resultado.isPresent()" before accessing the value. Bug Major Open Not assigned 10min effort Comment 1 month ago L135 cwe	
pedraAzul-back/.../appointments/services/UsuarioServiceTest.java	
☐ Call "resultado.isPresent()" before accessing the value. Bug Major Open Not assigned 10min effort Comment 1 month ago L68 cwe	
pedraAzul-back/.../appointments/shared/config/DataLoader.java	
☐ Remove this useless assignment to local variable "admin". Code Smell Major Open Not assigned 15min effort Comment 12 days ago L55 cert, cwe, unused	
☐ Remove this unused "admin" local variable. Code Smell Minor Open Not assigned 5min effort Comment 12 days ago L54 unused	
pedraAzul-back/.../appointments/shared/config/KeycloakMigrationRunner.java	
☐ Declare "omitidos" and all following declarations on a separate line. Code Smell Minor Open Not assigned 2min effort Comment 12 days ago L49 cert, convention	
☐ Define a constant instead of duplicating this literal "password" 3 times. Code Smell Critical Open Not assigned 8min effort Comment 12 days ago L75 design	
☐ Provide the parametrized type for this generic. Code Smell Major Open Not assigned 5min effort Comment 12 days ago L79 pitfall	
☐ Define a constant instead of duplicating this literal "/admin/realms/" 4 times. Code Smell Critical Open Not assigned 10min effort Comment 12 days ago L111 design	
☐ Provide the parametrized type for this generic. Code Smell Major Open Not assigned 5min effort Comment 12 days ago L145 pitfall	
pedraAzul-back/.../appointments/shared/config/SecurityConfig.java	
☐ Define a constant instead of duplicating this literal "ADMINISTRADOR" 15 times. Code Smell Critical Open Not assigned 32min effort Comment 19 days ago L41 design	
☐ Define a constant instead of duplicating this literal "AGENDADOR" 12 times. Code Smell Critical Open Not assigned 26min effort Comment 19 days ago L41 design	
☐ Define a constant instead of duplicating this literal "PACIENTE" 5 times. Code Smell Critical Open Not assigned 12min effort Comment 1 month ago L43 design	

piedraAzul-back/.../appointments/auth/dto/RegistroPacienteRequest.java	Remove this unused "fechaNacimiento" private field.	1 month ago ▾ L38
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
piedraAzul-back/.../appointments/cita/controller/CitaController.java	Remove this unused "usuarioService" private field.	1 month ago ▾ L38
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
piedraAzul-back/.../appointments/cita/dto/CitaRequest.java	Remove this unused "observacion" private field.	1 month ago ▾ L26
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
piedraAzul-back/.../appointments/cita/dto/CitaResponse.java	Remove this unused "id" private field.	1 month ago ▾ L14
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
	Remove this unused "medicoid" private field.	1 month ago ▾ L15
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
	Remove this unused "nombreMedico" private field.	1 month ago ▾ L16
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
	Remove this unused "pacienteld" private field.	1 month ago ▾ L17
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
	Remove this unused "nombrePaciente" private field.	1 month ago ▾ L18
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾
	Remove this unused "numeroDocumentoPaciente" private field.	1 month ago ▾ L19
☐	Code Smell ▾ Major ▾ Open ▾ Not assigned ▾ 5min effort Comment	unused ▾

Problema 3: Bugs en Pruebas Unitarias (Optional.get() sin isPresent())

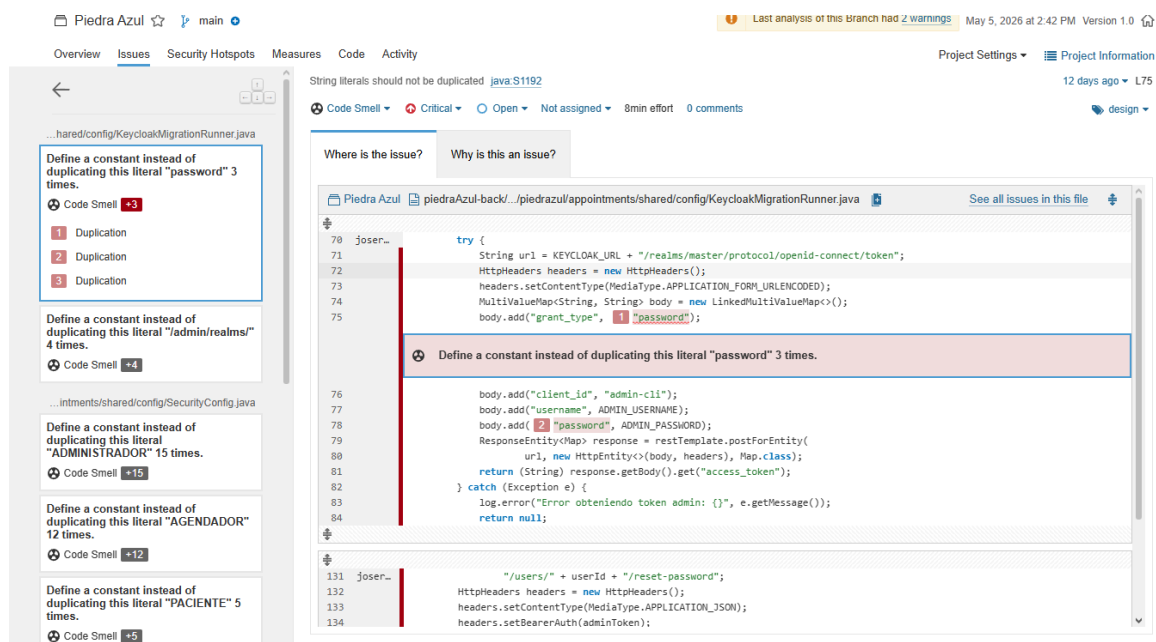
- **Descripción:** En `MedicoTerapeutaServiceTest.java` (L88) y `PacienteServiceTest.java` (L72), se intenta acceder al valor de un Optional directamente.
- **Impacto:** Esto genera un riesgo alto de NoSuchElementException. Si la prueba falla en la base de datos o en el mock, el test lanzará una excepción en lugar de un fallo controlado, lo que indica pruebas unitarias frágiles y poco fiables.

Problema 4: Violación del principio DRY (Define a constant instead of duplicating literal)



- **Descripción:** En `SecurityConfig.java` y `KeycloakMigrationRunner.java`, se repiten literales como "ADMINISTRADOR" (15 veces), "AGENDADOR" (12 veces).
- **Impacto:** Es un problema crítico de mantenibilidad. Si el nombre de un rol cambia, el equipo debe buscar y reemplazar manualmente en múltiples archivos, aumentando drásticamente la probabilidad de introducir errores tipográficos que rompan la seguridad del sistema.

Problema 5: Hard-coded Passwords



- **Descripción:** Se detectó la presencia de contraseñas quemadas directamente en el código fuente (literales como "password" o "PASSWORD").
- **Impacto:** Estas prácticas facilitan ataques de fuerza bruta o fugas de información sensible si el repositorio se ve comprometido.

4. ANÁLISIS CRÍTICO DE LA DEUDA TÉCNICA

Esta sección representa la evaluación de ingeniería sobre el estado actual del código, analizando por qué se generaron estas deficiencias y evaluando la precisión de la herramienta SonarQube.

Causas raíz

Tras el análisis, se identificó que los componentes con mayor acumulación de deuda técnica son **SecurityConfig.java** en el backend y **PortalPaciente.vue** en el frontend. El estado de estos módulos se atribuye a los siguientes factores:

- **Falta de diseño previo (Hardcoding y Accesibilidad):**
 - En el backend, se priorizó la velocidad de entrega "quemando" (hardcoding) literales de texto para roles y rutas de seguridad en lugar de centralizarlos.
 - En el frontend, el desarrollo se centró en la funcionalidad visual inmediata, omitiendo el uso de etiquetas semánticas y descripciones de tablas necesarias para cumplir con los estándares **WCAG 2.1**.
- **Violación de principios de Clean Code (DRY y SRP):**
 - La repetición masiva de cadenas de texto (ej. "ADMINISTRADOR" 15 veces) refleja una violación del principio *Don't Repeat Yourself* (DRY), lo que incrementa la fragilidad del sistema ante cambios de configuración.
 - La excesiva longitud y complejidad de componentes como **PortalPaciente.vue** sugiere una baja cohesión, donde una sola vista asume demasiadas responsabilidades de presentación y lógica.
- **Pruebas unitarias frágiles:**
 - La detección de bugs en los archivos de test (uso de `.get()` en **Optional** sin verificar `isPresent()`) evidencia que las pruebas se construyeron de forma apresurada, enfocándose en la cobertura técnica pero descuidando la robustez ante posibles fallos de datos.

Evaluación de falsos positivos

El equipo auditó los hallazgos de SonarQube e identificó los siguientes casos que, bajo el contexto específico del proyecto, no representan un riesgo real inmediato:

1. **Backend (Lógica de Persistencia):**
 - **Hallazgo:** "Remove this unused 'admin' local variable" en **DataLoader.java**.
 - **Justificación:** Se considera un **Falso Positivo temporal**. Esta variable captura el objeto persistido para una integración futura con el servicio de notificaciones por correo. Eliminarla ahora obligaría a reestructurar una lógica de carga de datos que ya ha sido validada funcionalmente.
2. **Frontend (Diseño de Interfaz):**
 - **Hallazgo:** "Add a description to this table" en **Configuracion.vue**.

- **Justificación:** Se marca como **Falso Positivo** debido a que dicha "tabla" no contiene datos tabulares de negocio, sino que se utilizó como un contenedor estructural para alinear elementos de un formulario complejo. Aunque no cumple con la semántica ideal, no afecta la interpretación de datos críticos por parte de un usuario con discapacidad. Sin embargo reconocemos que no es la mejor de las prácticas así que decidimos dejarla como una deuda técnica de diseño, y se prioriza su corrección en el siguiente sprint para migrar a Flexbox/Grid.

5. PLAN DE MITIGACIÓN Y REFACTORIZACIÓN

Hallazgo	Acción correctiva	Responsable
Field Injection en Servicios	Refactorizar hacia inyección por constructor para mejorar la testabilidad.	Jose Alejandro Rodriguez
Complejidad Ciclomática en Vue	Extraer lógica de los componentes hacia <i>Composables</i> (Composition API) o <i>Vuex/Pinia</i> .	Bibiana Sofia Cortes
Bugs de Confiabilidad (Reliability C)	Corregir los 7 bugs detectados, priorizando el manejo de <code>NullPointerException</code> potenciales.	Santiago Enrique Bastidas
Literales duplicados (Security)	Crear una clase de constantes o un <code>Enum</code> llamado <code>UserRoles</code> para centralizar "ADMINISTRADOR", "AGENDADOR", etc.	Bibiana Sofía Cortés
Bugs de Optional en Tests	Refactorizar <code>MedicoTerapistaServiceTest</code> y similares usando <code>.orElseThrow()</code> o verificando con <code>isPresent()</code> antes del <code>.get()</code> .	Jose Alejandro Rodriguez

Accesibilidad en Tablas (Front)	Agregar etiquetas <code><caption></code> y descripciones semánticas a las tablas de <code>PortalPaciente.vue</code> .	Bibiana Sofia Cortes
--	---	----------------------

CONCLUSIÓN

El proceso de análisis estático implementado mediante **SonarQube** ha permitido una auditoría profunda sobre la salud técnica del proyecto "Piedra Azul". A través de este ejercicio, se concluye lo siguiente:

- **Impacto de la Calidad en la Mantenibilidad:** Aunque el proyecto superó el *Quality Gate* por defecto, la presencia de **91 Code Smells** y un alto volumen de **literales duplicados** en la configuración de seguridad indica que la mantenibilidad a largo plazo está comprometida. La falta de centralización de constantes (DRY) incrementa exponencialmente el riesgo de errores en futuras fases de escalamiento.
- **Fiabilidad de las Pruebas:** Los bugs críticos detectados en la capa de pruebas unitarias (uso indebido de `Optional.get()`) revelan una **fragilidad estructural** en la estrategia de testing. Se concluye que una prueba que no gestiona correctamente la ausencia de datos no es una medida de calidad confiable, sino una fuente potencial de falsos negativos en el entorno de integración continua.
- **Compromiso con la Accesibilidad y Estándares:** Los hallazgos en el frontend resaltan la necesidad de integrar la accesibilidad (**WCAG 2.1**) como un requisito funcional desde el diseño, y no como una corrección posterior. La deuda técnica de 1 día y 1 hora identificada no es solo un indicador de tiempo, sino un llamado a mejorar la cohesión y el acoplamiento tanto en los DTOs del backend como en las vistas de Vue.js.

En definitiva, este análisis transforma la percepción del equipo sobre el código fuente: de ser un producto "funcional" a uno que debe ser **auditable, escalable y técnicamente sostenible**. El plan de mitigación propuesto representa el compromiso técnico inmediato para saldar la deuda técnica de mayor riesgo antes de la próxima entrega al cliente.

